

- During recovery, the system does not have to start from scratch; it can roll back to the last checkpoint and replay committed transactions.

Undo and Redo Operations

- Recovery involves two essential types of operations:
- **Undo:** Revert the effects of incomplete or uncommitted transactions.
- **Redo:** Reapply committed transactions that might not have been written to disk at the time of failure.
- The Write-Ahead Logging (WAL) principle ensures that log records describing changes are written to stable storage before the changes themselves are applied to the database.

4.6.1 Deferred Update (No-Undo/Redo Approach)

- In this approach, the system does not write changes to the database immediately.
- Changes are made in temporary memory (buffer).
- Only after a COMMIT is the data written to the database.

4.6.2 Immediate Update (Undo/Redo Approach)

- Changes are written to the database as they happen, even before a COMMIT.
- To ensure recoverability, logs are used to undo incomplete transactions and redo committed ones.

4.6.3 Shadow Paging

- Instead of modifying the actual database pages directly, the system maintains a shadow copy of the database.
- All operations are performed on a copy.
- Once the transaction successfully commits, the new version replaces the old version atomically.

CHAPTER 5 NoSQL CONCEPTS

5.1 INTRODUCTION TO NoSQL CONCEPTS

NoSQL (Not Only SQL) databases are designed to handle large volumes of unstructured and semi-structured data. Unlike traditional relational databases that rely on fixed schemas and tables, NoSQL offers flexible data models and supports horizontal scaling. This makes them well-suited for modern applications that require high performance, scalability, and the ability to manage diverse data types efficiently.

Key Features

- **Dynamic schema:** Allow flexible shaping of data to meet new requirements without the need to migrate or change schemas.
- **Horizontal scalability:** They scale horizontally for adding more nodes into the existing ones and acquire enough storage for even bigger datasets and much higher traffic by distributing the load on multiple servers.
- **Document-based:** Data are presented in flexible, semi-structured formats like JSON/BSON (e.g., MongoDB).
- **Key-value-based:** They possess a simple but fast access pattern (e.g., Redis) by storing data as pairs of keys and values.
- **Column-based:** Data are organized into columns instead of rows (e.g., CASSANDRA).
- **Distributed and high availability:** They are designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.

- **Flexibility:** Allow developers to store and retrieve data in a flexible and dynamic manner, with support for multiple data types and changing data structures.
- **Performance:** Perfect for big data and real-time analytics and high volume applications.

Challenges

- **Lack of standardization:** NoSQL systems can be vastly different from one another, making it even harder to choose the right one for a specific use case.
- **Lack of ACID compliance:** NoSQL databases may not provide consistency, which is a disadvantage for applications that need strict data integrity.
- **Narrow focus:** Great for storage but lack functionalities as transaction management, in which relational databases are great.
- **Absence of Complex Query Support:** They are not designed to handle complex queries, which means that they are not a good fit for applications that require complex data analysis or reporting.
- **Lack of maturity:** Being relatively new, NoSQL may not have the reliability, security and feature set of traditional relational databases.
- **Management complexity:** For large datasets, maintaining a NoSQL database could be quite more complicated than managing a relational database.
- **Limited GUI Tools:** While some NoSQL databases, like MongoDB offer GUI tools like MongoDB Compass, not all NoSQL databases provide flexible or user-friendly GUI tools.

SQL vs. NoSQL:

Feature	SQL(Relational DB)	NoSQL(Non-Relational DB)
Data Model	Structured, Tabular	Flexible (documents, key-value, graphs)
Scalability	Vertical Scaling	Horizontal Scaling
Schema	Predefined	Dynamic and schema-less
ACID support	Strong	Limited or Eventual Consistency
Best for	Transactional Applications	Big Data, real-time analytics
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

Popular NoSQL Databases & Their Use Cases

NoSQL Database	Type	UseCases
MongoDB	Document based	Content management, product catalogs
Redis	Key-value Store	Caching real-time analytics, session storage
Cassandra	Column- Family Store	Big data, high availability systems
Neo4j	Graph Database	Fraud detection, social networks

Use of NoSQL

- **Big Data Applications:** Efficiently stores and processes massive amounts of unstructured and semi-structured data.
- **Real-Time Analytics:** Supports fast queries and analysis for use cases like recommendation engines or fraud detection.
- **Scalable Web Applications:** Handles high traffic and large user bases by scaling horizontally across servers.
- **Flexible Data Storage:** Manages diverse data formats (JSON, key-value, documents, graphs) without rigid schemas.

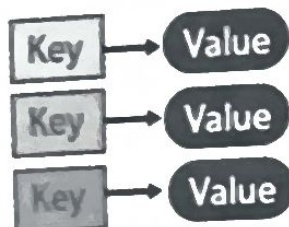
5.2 TYPES OF NoSQL DATABASES

NoSQL databases can be classified into four main types, based on their data storage and retrieval methods:

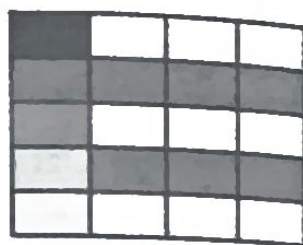
1. Document-based databases
2. Key-value stores
3. Column-oriented databases
4. Graph-based databases

NoSQL

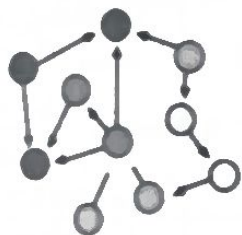
Key-Value



Column-Family



Graph



Document



5.2.1 Document-Based Database

The document-based database is a nonrelational database. Instead of storing the data in rows and columns (tables), it uses the documents to store the data in the database. A document database stores data in JSON, BSON or XML documents. Documents can be stored and retrieved in a form that is much closer to the data objects used in applications which means less translation is required to use these data in the applications. In the Document database, the particular elements can be accessed by using the index value that is assigned

for faster querying. Collections are the group of documents that store documents that have similar contents. Not all the documents are in any collection as they require a similar schema because document databases have a flexible schema.

Key features:

- **Flexible schema:** Documents in the database have a flexible schema. It means the documents in the database need not be the same schema.
- **Faster creation and maintenance:** The creation of documents is easy and minimal maintenance is required once we create the document.
- **No foreign keys:** There is no dynamic relationship between two documents so documents can be independent of one another. So, there is no requirement for a foreign key in a document database.
- **Open formats:** To build a document we use XML, JSON, and others.

Examples of document databases are MongoDB and Couchbase. A typical document will look like the following:

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```


Popular Document Databases & Use Cases

Database	Usecase
MogoDB	Content Management, Product Catalogs, User Profiles
CouchDB	Offline applications, mobile synchronization
Firebase Firestore	Rel-time apps, chat applications

5.2.2 Key-Value Stores

A key-value store is a nonrelational database. The simplest form of a NoSQL database is a key-value store. Every data element in the database is stored in key-value pairs. The data can be retrieved by using a unique key allotted to each element in the database. The values can be simple data types like strings, numbers or complex objects. A key-value store is like a relational database with only two columns which is the key and the value.

Key features:

- **Simplicity:** Data retrieval is extremely fast due to direct key access.
- **Scalability:** Designed for horizontal scaling and distributed storage.
- **Speed:** Ideal for caching and real-time applications.

Popular Key-Value Databases & Use Cases

Database	Use Case
Redis	Caching, real-time leaderboards, session storage
Memcached	High-speed in-memory caching
Amazon DynamoDB	Cloud-based scalable applications

Examples are Amazon DynamoDB and Redis. A simple view of data stored in a key-value database is given below:

Key: user:12345

Value: {"name": "foo bar", "email": "foo@bar.com", "designation": "software developer"}

5.2.3 Column Oriented Databases

A column-oriented database is a non-relational database that stores the data in columns instead of rows. That means when we want to run analytics on a small number of columns, we can read those columns directly without consuming memory with the unwanted data. Columnar databases are designed to read data more efficiently and retrieve the data with greater speed. A columnar database is used to store a large amount of data.

Key features :

- **High Scalability:** Supports distributed data processing.
- **Compression:** Columnar storage enables efficient data compression.
- **Faster Query Performance:** Best for analytical queries.

Popular Column-Oriented Databases & Use Cases

Database	Use Case
Apache Cassandra	Real-time analytics, IoT applications
Google Bigtable	Large-Scale machine learning, time-series data
HBase	Hadoop ecosystem, distributed storage

A typical example of how data is stored in a column-oriented database is as follows:

name	id	email	dob	city
Foo bar	12345	foo@bar.com		Some city
Carn Yale	34521	bar@foo.com	12-05-1972	

5.2.4 Graph-Based Databases

Graph-based databases focus on the relationship between the elements. It stores the data in the form of nodes in the database. The connections between the nodes are called links or relationships, making them ideal for complex relationship-based queries.

- Data is represented as nodes (objects) and edges (connections).
- Fast graph traversal algorithms help retrieve relationships quickly.
- Used in scenarios where relationships are as important as the data itself.

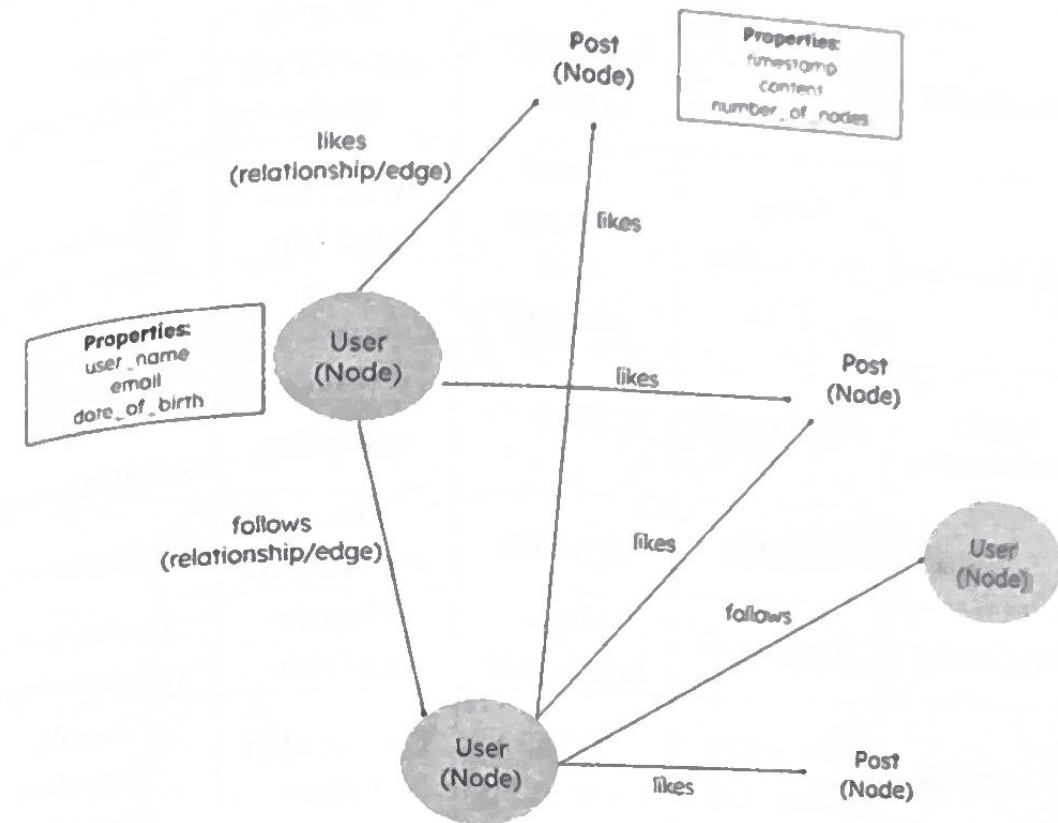
Key features :

- **Relationship-Centric Storage:** Perfect for social networks, fraud detection, recommendation engines.
- **Real-Time Query Processing:** Queries return results almost instantly.
- **Schema Flexibility:** Easily adapts to evolving relationship structures

Popular Graph Databases & Use Case:

Database	Usecase
Neo4j	Fraud detection, Social Networks
Amazon Neptune	Knowledge detection, Ai recommendations
ArangoDB	Multi-model database, cyber security

Below is an example of how data is stored:



Comparison of NoSQL Database Types

Key-value database	Document database	Wide-column database	Graph database
Basic support for data visualization	Basic support for data visualization	Basic support for data visualization	Basic support for data visualization
Basic authentication, limited access control	Role-based access control, encryption at-rest, encryption in transit, and in use	Role-based access control (RBAC), encryption at rest	Role-based access control (RBAC), encryption at rest, in transit
Limited support for online data archival	Automatic online data archival	Limited support for online data archival	Limited support for online data archival
Basic key-based data retrieval	Full-text search, vector search	Limited search capabilities	Limited search capabilities
Basic CRUD operations, limited data manipulation capabilities	Advanced query and data manipulation capabilities	Data manipulation and CRUD capabilities	Supports advanced graph traversal and manipulation operations
Stores any type of data	Stores JSON/BSON data	Columns and rows matrix	Nodes, edges, and relationships
Limited support for complex analytics	Suited for time-series, IoT analytics, real-time analytics	Suited for time-series data, IoT analytics	Well-suited for graph analytics
Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency
Simple indexes	Indexes on fields	Secondary indexes	Indexes on nodes, edges
Limited by key	Rich querying capability	Limited by columns	Specialized graph queries
Schematless	Flexible schema	Flexible schema	Flexible schema
Simple key-value pairs	JSON/BSON documents	Column and row families	Nodes, edges, and relationships

Feature	Document Based	Key Value Store	Column Oriented	Graph Based
Data Model	JSON-like documents	Key-Value pairs	Columns instead of rows	Nodes and Relationships
Best Use Case	Semi structured data	Fast lookups and caching	Analytics and big data	Relationship - heavy data
Query Performance	Moderate	Fast	High for analytics	Optimized for relationships
Schema	Flexible	Dynamic	Semi-structured	Schema-less
Scalability	Horizontal	High horizontal	Highly scalable	Scales with relationships
Examples	MongoDB, CouchDB	Edis, DyamoDB	Cassandra, HBase	Neo4j, Amazon Neptune

5.3 CAP THEOREM

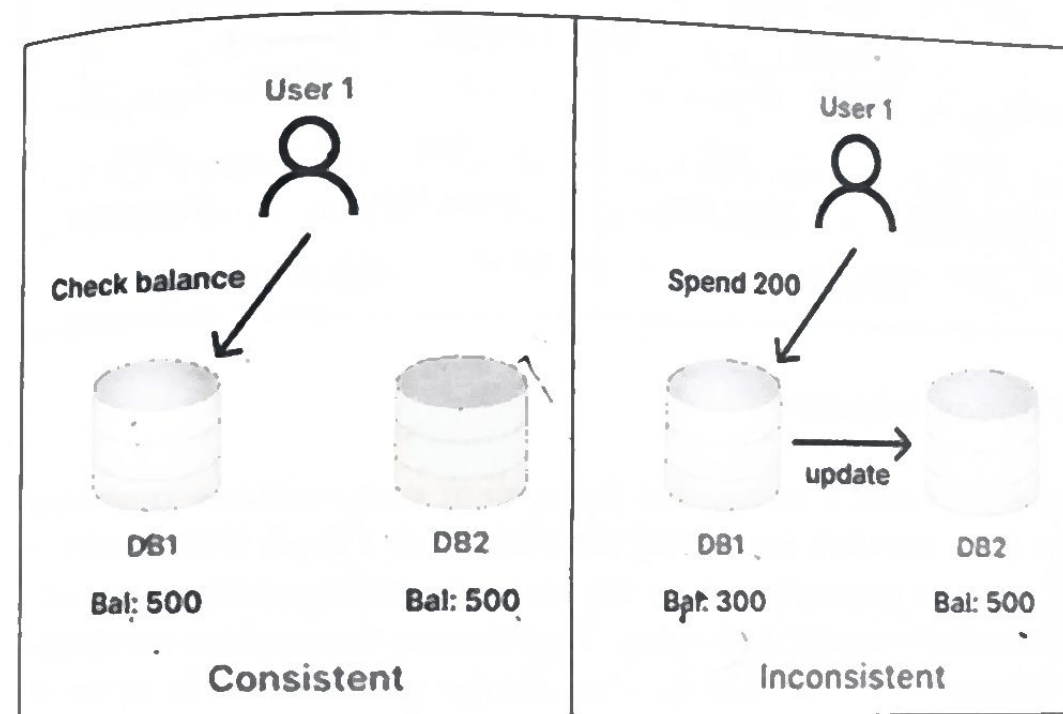
The CAP theorem emphasizes the limitations that system designers have while addressing distributed data replication. It states that only two of the three properties – consistency, availability, and partition tolerance can be concurrently attained by a distributed system. Developers must carefully balance these attributes according to their particular application demands because of this underlying restriction. They may decide which qualities to prioritize to obtain the best performance and reliability for their systems by knowing the CAP theorem. It asserts that all three of the following qualities cannot be concurrently guaranteed in any distributed data system:

1. Consistency

Consistency means that all the nodes (databases) inside a network will have the same copies of a replicated data item visible for various transactions. It guarantees that every node in a distributed cluster returns the same, most recent, and successful write. It refers to every client having the same view of the data. There are various types of consistency models. Consistency in CAP refers to sequential

consistency, a very strong form of consistency. The concept of Consistency in ACID and CAP are slightly different since in CAP, it refers to the consistency of the values in different copies of the same data item in a replicated distributed system. In ACID, it refers to the fact that a transaction will not violate the integrity constraints specified on the database schema.

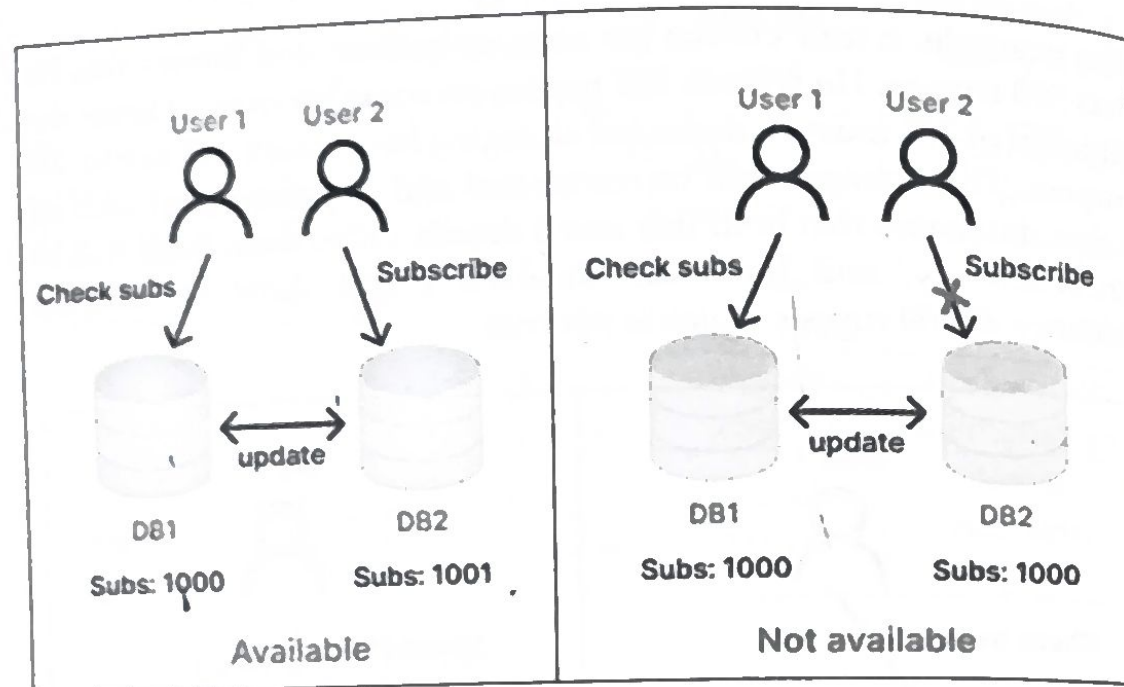
For example, a user checks his account balance and knows that he has 500 rupees. He spends 200 rupees on some products. Hence the amount of 200 must be deducted changing his account balance to 300 rupees. This change must be committed and communicated with all other databases that hold this user's details. Otherwise, there will be inconsistency, and the other database might show his account balance as 500 rupees which is not true.



2. Availability

Availability means that each read or write request for a data item will either be processed successfully or will receive a message that the operation cannot be completed. Every non-failing node returns a response for all the read and write requests in a reasonable amount of time. The key word here is "every". In simple terms, every node (on either side of a network partition) must be able to respond in a reasonable amount of time.

For example, user1 is a content creator having 1000 other users subscribed to his channel. Another user 2 who is far away from user1 tries to subscribe to user 1's channel. Since the distance between both users are huge, they are connected to different database nodes of the social media network. If the distributed system follows the principle of availability, user2 must be able to subscribe to user 1's channel.

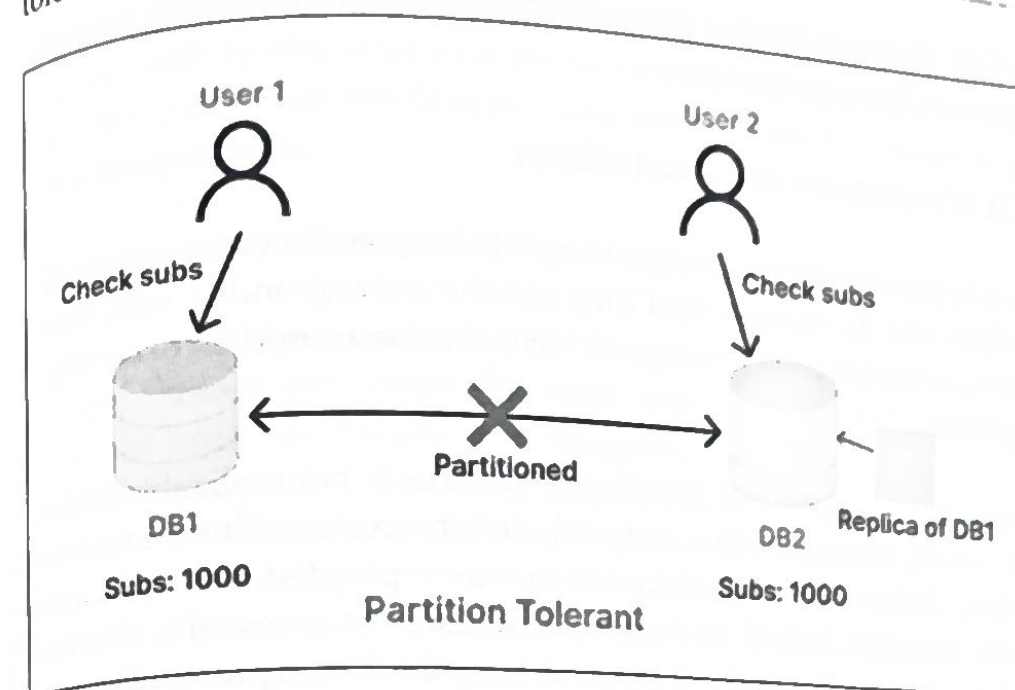


3. Partition Tolerance

Partition tolerance means that the system can continue operating even if the network connecting the nodes has a fault that results in two or more partitions, where the nodes in each partition can only communicate among each other. That means, the system continues to function and upholds its consistency guarantees in spite of network partitions. Distributed systems guaranteeing partition tolerance can gracefully recover from partitions once the partition heals.

For example, The same social media network where two users are trying to find the subscriber count of a particular channel. Due to some technical fault, there occurs a network outage, the second database connected by user 2 loses its connection with the first database. Hence the subscriber count is shown to user 2 with the help of replicas of data which was previously stored in database 1 backed

up prior to network outage. Hence the distributed system is partition tolerant.



The CAP theorem states that distributed databases can have at most two of the three properties: consistency, availability, and partition tolerance. As a result, database systems prioritize only two properties at a time.

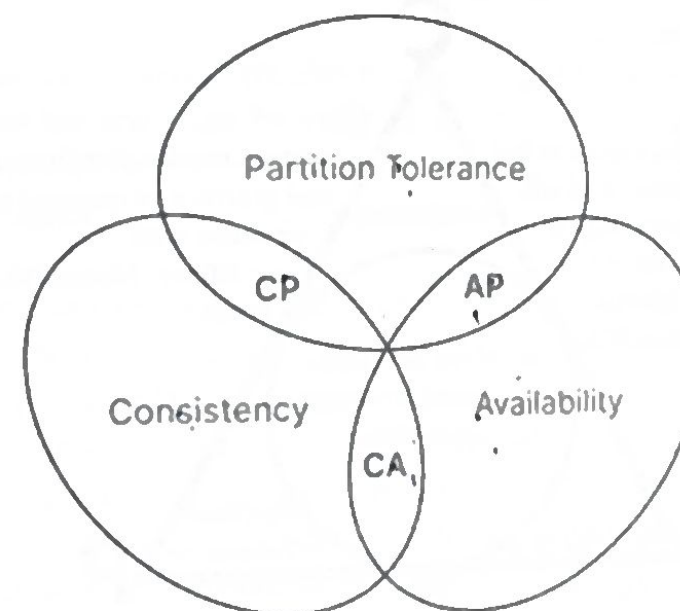


Fig: Venn diagram of CAP theorem

The Trade-Offs in the CAP Theorem

The CAP theorem implies that a distributed system can only provide two out of three properties:

1. CA (Consistency and Availability)

These types of systems always accept the request to view or modify the data sent by the user and they are always responded with data which is consistent among all the database nodes of a big, distributed network.

However, such a type of distributed systems is not realizable in the real world because when network failure occurs, there are two options: Either send old data which was replicated moments ago before network failure or do not allow users to access the already existing data (old data). If users choose the first option, his/ her system will become Available and if users choose the second option our system will become Consistent.

Example databases: MySQL, PostgreSQL

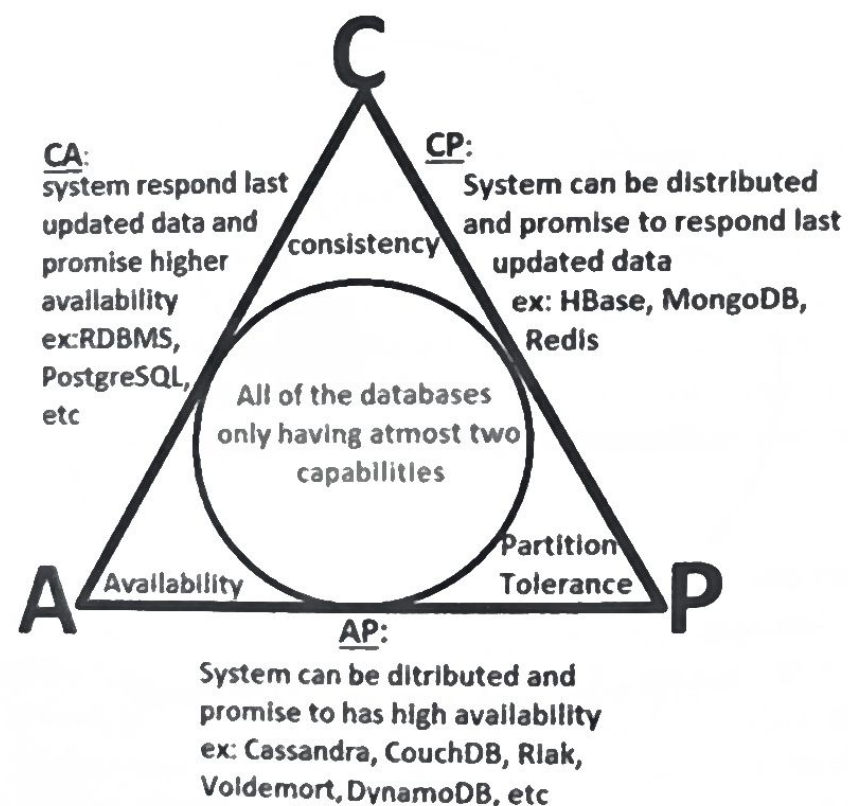


Fig: CAP diagram

2. AP (Availability and Partition Tolerance)

These types of system are distributed in nature, ensuring that the request sent by the user to view or modify the data present in the database nodes are not dropped and are processed in presence of a network partition.

The system prioritizes availability over consistency and can respond with possibly stale data which was replicated from other nodes before the partition was created due to some technical failure. Such design choices are generally used while building social media websites such as Facebook, Instagram, Reddit, etc. and online content websites like YouTube, blog, news, etc. where consistency is usually not required, and a bigger problem arises if the service is unavailable causing corporations to lose money since the users may shift to a new platform. The system can be distributed across multiple nodes and is designed to operate reliably even in the face of network partitions.

Example databases: Amazon DynamoDB, Google Cloud Spanner.

3. CP (Consistency and Partition Tolerance)

These types of systems are distributed in nature, ensuring that the request sent by the user to view or modify the data present in the database nodes are dropped instead of responding with inconsistent data in presence of a network partition.

The system prioritizes consistency over availability and does not allow users to read crucial data from the stored replica which was backed up prior to the occurrence of network partition. Consistency is chosen over availability for critical applications where latest data plays an important role such as stock market application, ticket booking application, banking, etc. where problems will arise due to old data present to users of application.

For example, in a train ticket booking application, there is one seat which can be booked. A replica of the database is created, and it is sent to other nodes of the distributed system. A network outage occurs which causes the user connected to the partitioned node to fetch details from this replica. Some users connected to the

unpartitioned part of the distributed network and already booked the last remaining seat. However, the user connected to the partitioned node will still have one seat which makes the available data inconsistent. It would have been better if the user was shown an error and made the system unavailable for the user and maintain consistency. Hence consistency is chosen in such scenarios.

Example databases: Apache HBase, MongoDB, Redis.

5.4 BASE PROPERTIES

The BASE properties of a database management system are a set of principles that guide the design and operation of modern databases. The acronym BASE stands for Basically Available, Soft State, and Eventual Consistency.

Basically Available

This property refers to the fact that the database system should always be available to respond to user requests, even if it cannot guarantee immediate access to all data. The database may experience brief periods of unavailability, but it should be designed to minimize downtime and provide quick recovery from failures.

Soft State

This property refers to the fact that the state of the database can change over time, even without any explicit user intervention. This can happen due to the effects of background processes, updates to data, and other factors. The database should be designed to handle this change gracefully, and ensure that it does not lead to data corruption or loss.

Eventual Consistency

This property refers to the eventual consistency of data in the database, despite changes over time. In other words, the database should eventually converge to a consistent state, even if it takes some time for all updates to propagate and be reflected in the data. This is in contrast to the immediate consistency required by traditional ACID-compliant databases.

Uses of BASE Databases

BASE databases are used in modern, highly-available, and scalable systems that handle large amounts of data. Examples of such systems include online shopping websites, social media platforms, and cloud-based services.

Difference between Base Properties and ACID Properties

ACID	BASE
ACID (Atomicity, Consistency, isolation, Durability) is a set of properties that guarantee the integrity and consistency of data in a traditional database.	The BASE properties are a more relaxed version of ACID that trade off some consistency guarantees for greater scalability and availability.
The primary difference between the two is that ACID requires immediate consistency	While BASE only requires eventual consistency.
ACID is better suited to traditional transactional databases.	The BASE is more suitable for use in large-scale, highly-available systems

5.5 USE CASES AND LIMITATIONS OF NoSQL

Use Cases for NoSQL Databases

- **Big Data and Real-Time Analytics:** NoSQL databases are ideal for big data and real-time analytics applications that require scalable
- **Content Management and Personalization:** NoSQL databases are well-suited for content management systems, recommendation engines, and personalization platforms that handle semi-structured or unstructured data, such as user profiles, preferences, and interactions.

Limitations

- **Consistency Trade-offs:** Some NoSQL databases sacrifice strong consistency for improved availability and partition tolerance, as per the CAP theorem (Consistency, Availability, Partition tolerance). Depending on the chosen

consistency model (e.g., eventual consistency), developers may need to handle conflicts or stale data in distributed environments.

- **Limited ACID Support:** Not all NoSQL databases provide full ACID compliance, which may be a requirement for certain mission-critical applications. While some NoSQL databases offer transactional support for specific use cases, developers need to carefully evaluate the trade-offs between consistency, availability, and performance.
- **Complexity in Data Modeling:** NoSQL databases require careful consideration of data modeling and query patterns to optimize performance and scalability. Developers must design schemas and access patterns based on the application's requirements, which can be challenging without prior experience with NoSQL technologies.
- **Tooling and Ecosystem Maturity:** Compared to SQL databases, the tooling and ecosystem for NoSQL databases may be less mature, with fewer third-party libraries, integrations, and community resources available. This can pose challenges for developers in terms of debugging, monitoring, and maintaining NoSQL-based applications.
- **Learning Curve:** Transitioning from SQL to NoSQL databases may involve a learning curve for developers, particularly those accustomed to relational database paradigms. NoSQL databases introduce new concepts, data models, and query languages that require adaptation and specialized expertise for effective usage.

CHAPTER 6 SQL ARCHITECTURAL PATTERNS

6.1 SQL ARCHITECTURAL PATTERNS

Architecture Pattern is a logical way of categorizing data that will be stored on the Database. NoSQL is a type of database which helps to perform operations on big data and store it in a valid format. It is widely used because of its flexibility and a wide variety of services.

Architecture Patterns of NoSQL:

The data is stored in NoSQL in any of the following four data architecture patterns.

1. Key-Value Store Database
2. Column Store Database
3. Document Database
4. Graph Database

These are explained below.

6.2 KEY-VALUE STORE DATABASE

This model is one of the most basic models of NoSQL databases. As the name suggests, the data is stored in the form of Key-Value Pairs. The key is usually a sequence of strings, integers or characters but can also be a more advanced data type. The value is typically linked or co-related to the key. The key-value pair storage databases generally store data as a hash table where each key is unique. The

value can be of any type (JSON, BLOB(Binary Large Object), strings, etc). This type of pattern is usually used in shopping websites or e-commerce applications.

Advantages

- Can handle large amounts of data and heavy load
- Easy retrieval of data by keys.

Limitations

- Complex queries may attempt to involve multiple key-value pairs which may delay performance.
- Data can involve many-to-many relationships which may collide.

Examples

- DynamoDB
- Berkeley DB

Key:1	ID:210		
Key:2	ID:411	Email: geeksforgeeks@gmail.com	
Key:3	UID:219	Name: Geek	Age:20

6.3 COLUMN STORE DATABASE

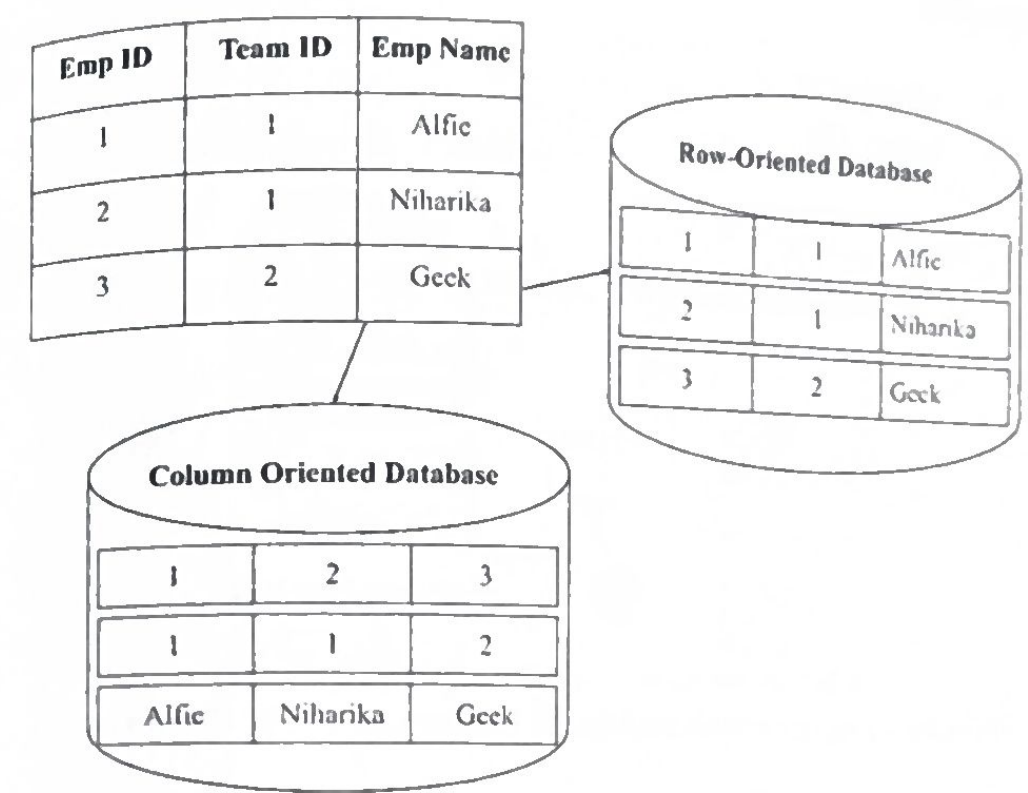
Rather than storing data in relational tuples, the data is stored in individual cells which are further grouped into columns. Column-oriented databases work only on columns. They store large amounts of data into columns together. Format and titles of the columns can diverge from one row to another. Every column is treated separately. But still, each individual column may contain multiple other columns like traditional databases. Basically, columns are mode of storage in this type.

Advantages

- Data is readily available
- Queries like SUM, AVERAGE, COUNT can be easily performed on columns.

Examples

- HBase
- Bigtable by Google
- Cassandra



6.4 DOCUMENT DATABASE

The document database fetches and accumulates data in form of key-value pairs but here, the values are called as Documents. Document can be stated as a complex data structure. Document here can be a form of text, arrays, strings, JSON, XML or any such format. The use of nested documents is also very common. It is very effective as most of the data created is usually in the form of JSONs and is unstructured.

Advantages

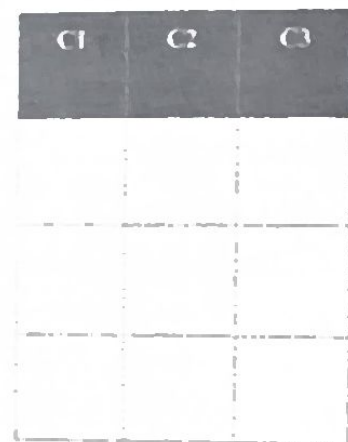
- This type of format is very useful and apt for semi-structured data.
- Storage retrieval and managing of documents is easy.

Limitations

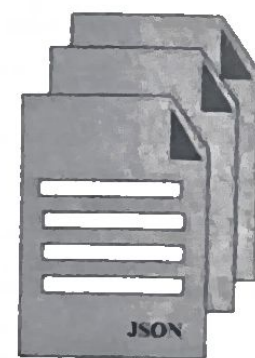
- Handling multiple documents is challenging
- Aggregation operations may not work accurately.

Examples

- MongoDB
- CouchDB



Relational Data Model



Document Store Model

Figure - Document Store Model in form of JSON documents

6.5 GRAPH DATABASES

This architecture pattern deals with the storage and management of data in graphs. Graphs are basically structures that depict connections between two or more objects in some data. The objects or entities are called nodes and are joined together by relationships called Edges. Each edge has a unique identifier. Each node serves as a point of contact for the graph. This pattern is very commonly used in social networks where there are a large number of entities and each entity has one or many characteristics which are connected by edges. The relational database pattern has tables that are loosely

connected, whereas graphs are often very strong and rigid in nature.

Advantages

- Fastest traversal because of connections.
- Spatial data can be easily handled.

Limitations

Wrong connections may lead to infinite loops.

Examples

- Neo4J
- FlockDB(Used by Twitter)

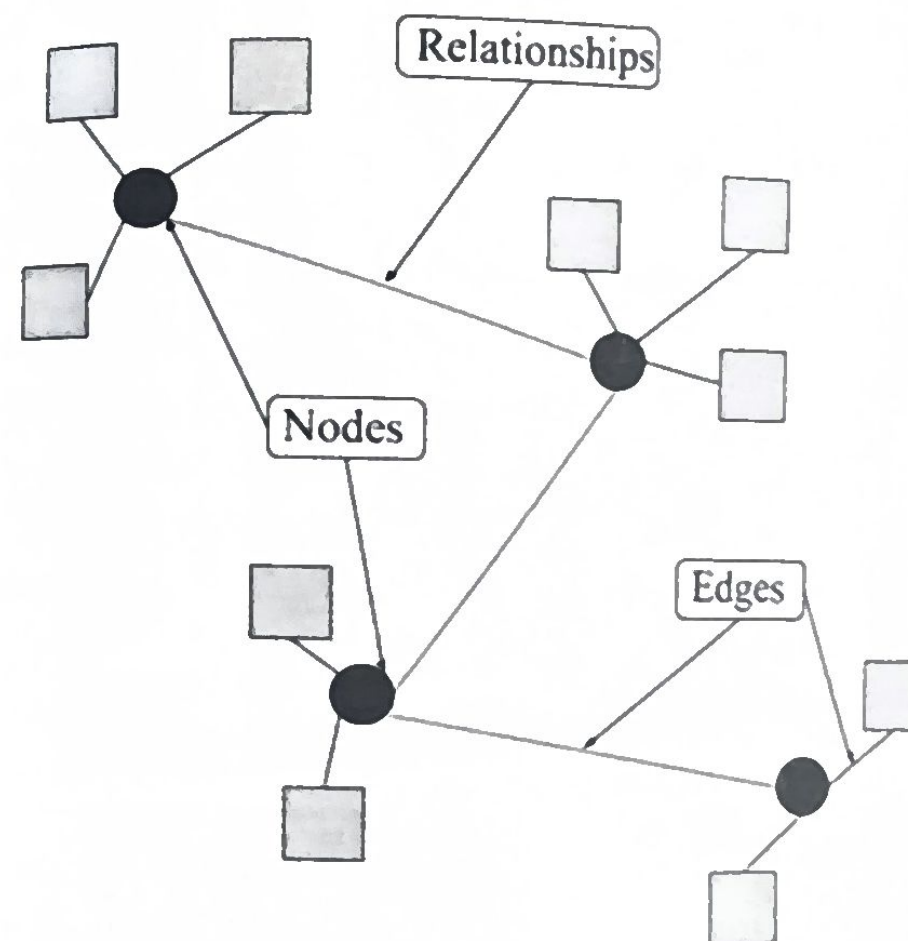
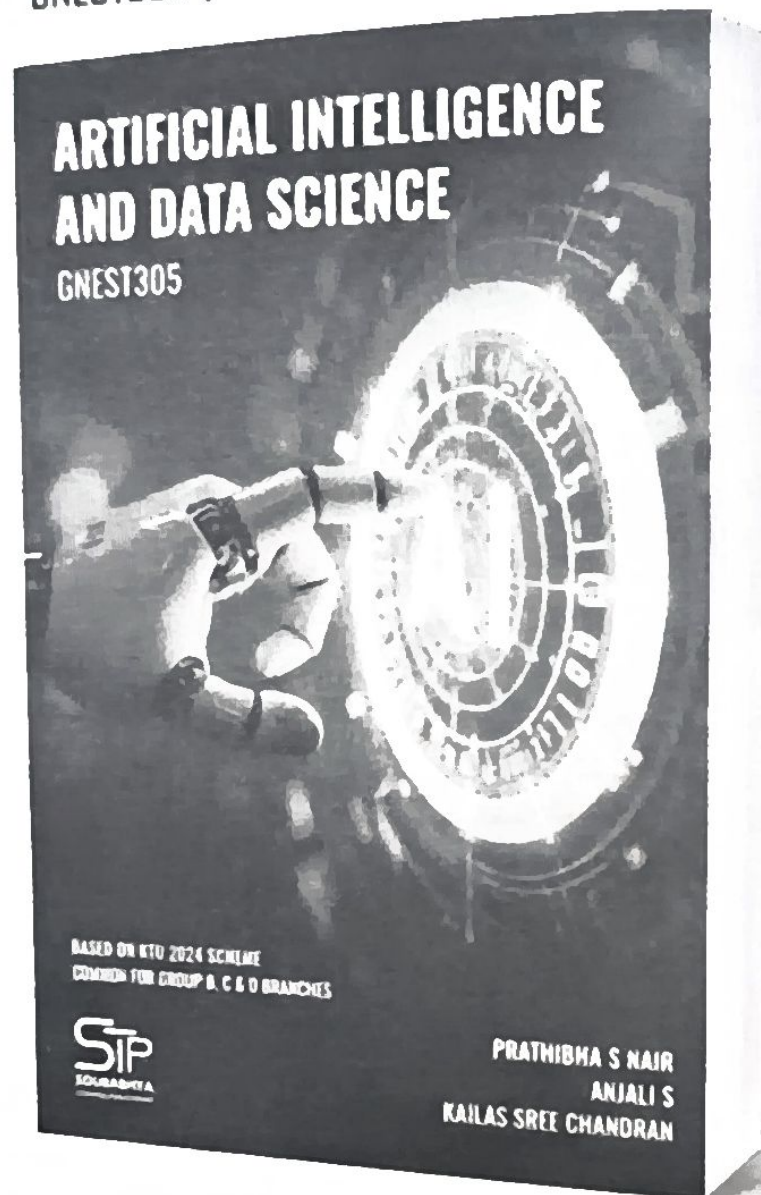


Figure - Graph model format of NoSQL Databases

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SEMESTER 3 - B.TECH KTU - 2024 SCHEME
GNEST305 (COMMON FOR GROUP B C D BRANCHES)



SCAN TO PURCHASE
WITH FREE SHIPPING



SCAN TO PURCHASE
FROM AMAZON

CHAPTER 7 TIMESTAMP BASED CONCURRENCY CONTROL

Timestamp-based concurrency control is a technique used in database management systems (DBMS) to ensure serializability of transactions without using locks. It uses timestamps to determine the order of transaction execution and ensures that conflicting operations follow a consistent order. Each transaction T is assigned a unique timestamp $TS(T)$ when it enters the system. This timestamp determines the transaction's place in the execution order.

TIMESTAMP ORDERING PROTOCOL

The Timestamp Ordering Protocol enforces that older transactions (with smaller timestamps) are given higher priority. This prevents conflicts and ensures the execution is serializable and deadlock-free.

For example:

- If Transaction T_1 enters the system first, it gets a timestamp $TS(T_1) = 007$ (assumption).
- If Transaction T_2 enters after T_1 , it gets a timestamp $TS(T_2) = 009$ (assumption).

This means T_1 is "older" than T_2 and T_1 should execute before T_2 to maintain consistency.